Linguagem de Programação

Professor: Euclides Paim
euclidespaim@gmail.com

Introdução à Lógica de Programação

Algoritmos e Entrada e Saída de Dados

Professor: Euclides Paim

euclidespaim@gmail.com

Objetivos da Aula

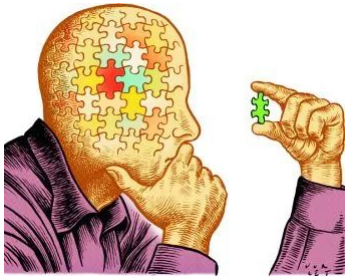
- Compreender o que é lógica de programação
- Conhecer seu contexto histórico
- Explorar conceitos fundamentais
- Resolver problemas simples
- Iniciar o estudo de algoritmos em Python

O que é Lógica de Programação?

- A lógica de programação é o conjunto de técnicas e raciocínios utilizados para resolver problemas através de sequências de instruções organizadas.

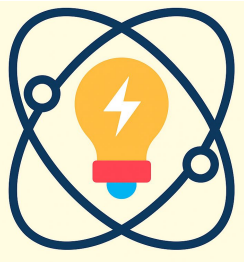
Por que aprender lógica de programação?

- Desenvolve o pensamento lógico
- Facilita o aprendizado de qualquer linguagem de programação
- Melhora a capacidade de resolver problemas



Contexto Histórico

- Lógica: origem na filosofia (Aristóteles)
- Lógica matemática: Leibniz, Boole
- Computação: Ada Lovelace, Alan Turing
- Lógica computacional moderna: base dos algoritmos



Ada Lovelace

- A primeira programadora da história.
- Criou algoritmos para a máquina analítica de Charles Babbage, no século XIX.

Alan Turing

- Matemático britânico que formalizou o conceito de algoritmo e máquina computacional.
- Turing idealizou o conceito de **máquina de Turing**, um modelo teórico que fundamenta a ciência da computação.
- Esse modelo é usado até hoje para entender o que é computável e como algoritmos funcionam.

Conceitos Fundamentais

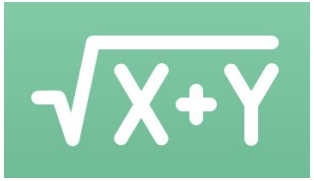
- Dados
- Variáveis
- Operadores
- Expressões
- Instruções
- Estruturas de controle.

Dados

- Dados são informações brutas, como números, textos ou sinais, que ainda não foram interpretados.
- Podem estar em diferentes formatos: tabelas, planilhas, sensores, arquivos, etc.
- Servem como matéria-prima para gerar conhecimento por meio da análise e processamento.

Variáveis

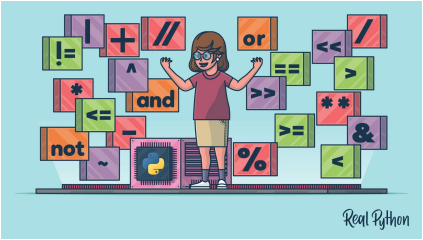
- Variáveis são **espaços na memória** usados para guardar dados que podem mudar durante a execução de um programa. Exemplo: `idade = 17`.
- Cada variável possui um **nome** (identificador) e, em muitas linguagens, um **tipo** (como número, texto, etc.).
- Permitem **reutilizar** e manipular valores sem precisar repetir o código, tornando o programa mais organizado e flexível.



Operadores

Símbolos usados para manipular valores:

- Aritméticos: `+ - * /`
- Relacionais: `== != > <`
- Lógicos: `and or not`



Expressões

- Combinação de variáveis, operadores e valores que produzem um novo valor.
  Exemplo: `nota_final = (nota1 + nota2) / 2`
- Podem ser Aritméticas, Lógicas e Literais.

Instruções

- Comandos que o computador **executa** para realizar uma ação específica, como mostrar algo na tela ou fazer um cálculo. Exemplo em pseudocódigo:
  `escreva "Digite um número"`
- Programas são compostos por várias instruções que são executadas em sequência, uma após a outra.
- Algumas instruções controlam o caminho que o programa segue, como `if`, `for`, `while`, permitindo **decisões** e **repetições**.

Estruturas de Controle

As **Estruturas de Controle** são comandos que permitem desviar o fluxo sequencial de um programa, possibilitando que ele tome decisões e execute blocos de código repetidamente. Elas são a base para a lógica de programação, pois controlam a ordem em que as instruções são executadas.

Existem dois tipos principais de estruturas de controle:

- **Estruturas de Decisão/Condicionais (e.g., `if`, `else`, `switch`):**
  Permitem que o programa execute um bloco de código **uma única vez (ou nenhuma)**, dependendo se uma condição lógica é verdadeira ou falsa.

- **Estruturas de Repetição/Laços (e.g., `for`, `while`, `do-while`):**
  Permitem que um bloco de código seja executado **múltiplas vezes** (em *loop*) enquanto (ou até que) uma condição específica seja atendida.

Pensamento Computacional

- Habilidade de resolver problemas de forma lógica e estruturada, como um computador faria.
- Inclui técnicas como **decomposição** (quebrar problemas em partes menores), **reconhecimento de padrões**, **abstração** e **algoritmos**.
- É uma habilidade útil em várias áreas, não só na computação, pois ajuda a **pensar com clareza**, planejar soluções e **tomar decisões** com base em dados.

Algoritmo – Definição

- Um algoritmo é uma **sequência ordenada de passos** que resolve um problema ou executa uma tarefa.
- Não precisa ser escrito em linguagem de programação — pode ser descrito em português, fluxograma ou pseudocódigo.
- Todo algoritmo precisa ser **bem definido** (sem ambiguidade) e **terminar** após um número finito de etapas.

Exemplo de Algoritmo (Diário)

Problema: ir para a escola
Passos:

1. Acordar
2. Escovar os dentes
3. Tomar café
4. Pegar a mochila
5. Ir para a escola

Características de um bom algoritmo

- Clareza
- Precisão
- Finitude
- Eficiência

Formas de Representar Algoritmos

- Linguagem natural
- Fluxogramas
- Pseudocódigo
- Linguagens de programação (ex: Python)

Exemplo de Pseudocódigo

```
inicio
  escreva "Digite um número"
  leia numero
  escreva "O número é", numero
fim
```

Exercício em Pseudocódigo

Crie um algoritmo que:

- Peça dois números
- Some-os
- Exiba o resultado

Representação em Fluxograma

- Apresente um exemplo visual do exercício anterior com blocos de:
  a. início,
  b. entrada,
  c. processamento,
  d. saída e
  e. fim.

Linguagens de Programação

- Ferramentas que transformam algoritmos em código que pode ser executado por computadores.
  Exemplos: Python, Java, C, JavaScript
- Linguagens de programação são usadas para **escrever algoritmos** que o computador possa entender e executar.
- O que escrevemos na linguagem é chamado de **código fonte**, que depois é interpretado ou compilado para virar uma instrução compreensível pela máquina.

Programação de Aplicativos

Introdução à Algoritmos

Python é uma linguagem de programação popular. Ela foi criada por Guido van Rossum e lançada em 1991.

É usada para:

- desenvolvimento web (lado do servidor),
- desenvolvimento de software,
- matemática,
- scripts de sistema.

O que o Python pode fazer?

- Python pode ser usado em um servidor para criar aplicações web.
- Python pode ser usado junto a softwares para criar fluxos de trabalho.
- Python pode se conectar a sistemas de banco de dados. Ele também pode ler e modificar arquivos.
- Python pode ser usado para lidar com big data e realizar cálculos matemáticos complexos.
- Python pode ser usado para prototipagem rápida ou para o desenvolvimento de software pronto para produção.

Primeiro Algoritmo em Python

```python
print("Digite um número:")
numero = input()
print("Você digitou:", numero)
```

Entrada de Dados

```
nome = input("Digite seu nome: ")
```

Saída de Dados

```
print("Olá,", nome)
```

Exercício Prático

Escreva um programa em Python que:

- Peça dois números
- Some-os
- Mostre o resultado

Dica para os Estudantes

- Praticar é a chave!
- Teste, erre, corrija e tente de novo.
- Programar é como aprender uma nova linguagem.

Próximos Passos

- Praticar lógica com exercícios
- Estudar mais algoritmos
- Começar a explorar estruturas de decisão e repetição em Python

Referências

Referências Básicas

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. ***Algoritmos: teoria e prática*.** 3. ed. Rio de Janeiro: Elsevier, 2013.
SEBESTA, Robert W. ***Conceitos de linguagens de programação***. 10. ed. São Paulo: Pearson, 2018.
MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. ***Algoritmos: lógica para desenvolvimento de programação de computadores*.** 27. ed. São Paulo: Érica, 2016.
DOWNEY, Allen B. ***Pense em Python: como pensar como um cientista da computação***. Tradução de Cássio de Souza Costa. 2. ed. São Paulo: Novatec, 2016.

Referências Complementares

IEPSEN, Edécio Fernando. ***Lógica de programação e algoritmos com JavaScript***. 2. ed. São Paulo: Novatec, 2022.

Referências na Internet

https://www.w3schools.com/python/